

408 数据结构

综合笔记

以历年真题为切入点，结合完整C代码，系统总结

覆盖全邀8章知识点 | 完整C代码实现 | 408真题标注

知识框架：绪论 / 线性表 / 栈和队列 / 串 / 树与二叉树 / 图 / 查找 / 排序

附录：大题模板 . 复杂度速查表

2026年6月 整理

目录

第1章 绪论

复杂度分析是一切的基础

408真题：2011-2014, 2017, 2019, 2022, 2025

第2章 线性表

顺序表与链表，一切数据结构之母

408真题：2010-2025 多题

第3章 栈和队列

受限的线性表，无限的考点

408真题：括号匹配, 表达式求值

第4章 串

KMP算法与模式匹配

408真题：KMP大题必考

第5章 树与二叉树

BST, AVL, 堆, Huffman

408真题：遍历, BST删除, AVL旋转

第6章 图

DFS/BFS, 最短路径, MST

408真题：Dijkstra, 拓扑排序, 关键路径

第7章 查找

二分查找, B树, 哈希表

408真题：ASL计算, B树分裂

第8章 排序

八大内部排序全解

408真题：快排, 堆排, 归并, 基数

附录A 大题模板

408手写算法必背模板

链表逆置, 归并, 遍历, 二分查找

附录B 复杂度速查

各结构/算法时空复杂度总表

408高频陷阱汇总

第1章 绪论

408真题年份：2011-2014, 2017, 2019, 2022, 2025

1.1 408真题引入

[2022年408真题] 分析程序段时间复杂度

`for(i=1;i<n;i*=2) for(j=0;j<i;j++) sum++;` 答案： $O(n)$

[2019年408真题] 分析while循环时间复杂度

`while(n>=(x+1)*(x+1)) x=x+1;` 答案： $O(\sqrt{n})$

[2017年408真题] 分析递归函数时间复杂度

`int func(int n){ while(sum<n) sum+=++i; }` 答案： $O(n^{1/2})$

1.2 核心知识点

1.2.1 时间复杂度分析

时间复杂度描述算法执行时间随问题规模 n 增大而变化的趋势。使用大 O 记号表示渐近上界。常见复杂度排序：

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

1.2.2 空间复杂度分析

空间复杂度 $S(n)$ 表示算法运行过程中所需的额外存储空间。

原地算法： $S(n)=O(1)$ ；递归空间=递归深度 $\times$ 每层空间。

1.3 完整C代码示例

```
// O(1) - constant
int getFirst(int a[], int n) { return a[0]; }

// O(n) - linear
int sum(int a[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}

// O(n^2) - quadratic
void printPairs(int a[], int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            printf("%d %d\n", a[i], a[j]);
}
```

```
// O(log n) - logarithmic
int binarySearch(int a[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

// O(n log n)
void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r); // O(n)
    }
}
```

1.4 递归复杂度递推公式

递推公式	结果	典型算法
$T(n)=T(n-1)+O(1)$	$O(n)$	阶乘、线性递归
$T(n)=T(n/2)+O(1)$	$O(\log n)$	二分查找
$T(n)=2T(n/2)+O(n)$	$O(n \log n)$	归并排序
$T(n)=T(n-1)+O(n)$	$O(n^2)$	冒泡/选择排序
$T(n)=2T(n-1)+O(1)$	$O(2^n)$	汉诺塔、递归Fibonacci

1.5 408易错点

- $\text{for}(i=1; i \leq n; i*=2)$ 是 $O(\log_2 n)$, 不是 $O(\sqrt{n})$
- 递归 Fibonacci 时间 $O(2^n)$, 但记忆化后 $O(n)$
- 空间复杂度不等于代码中变量个数(递归的调用栈也算入空间)
- $\text{for}(i=0; i < n; i++) \text{for}(j=i; j < n; j++)$ 总次数 $n(n+1)/2 \rightarrow O(n^2)$

第2章 线性表

408真题年份：2010-2025 多题涉及

2.1 408真题引入

[2023年408真题] 顺序表与链表的综合操作

考察顺序表插入删除, 链表逆置等核心操作。

[2021年408真题] 双指针法删除顺序表元素

删除顺序表中所有值为x的元素, 要求 $O(n)$ 时间, $O(1)$ 空间。

2.2 顺序表

顺序表用数组存储元素, 支持随机存取($O(1)$), 插入/删除需移动元素($O(n)$)。

```
// ██████████
#define INIT_SIZE 100
#define INCREMENT 10

typedef struct {
    int *elem;      // ████
    int length;     // █████
    int capacity;   // █████
} SeqList;

// ████  $O(1)$ 
void InitList(SeqList *L) {
    L->elem = (int*)malloc(INIT_SIZE * sizeof(int));
    L->length = 0;
    L->capacity = INIT_SIZE;
}

// ███  $O(n)$  -- █████  $n/2$  ████
int ListInsert(SeqList *L, int i, int e) {
    if (i < 1 || i > L->length + 1) return 0;
    if (L->length >= L->capacity) {
        L->elem = (int*)realloc(L->elem,
                                (L->capacity + INCREMENT) * sizeof(int));
        L->capacity += INCREMENT;
    }
    for (int j = L->length; j >= i; j--)
        L->elem[j] = L->elem[j - 1];
    L->elem[i - 1] = e;
    L->length++;
    return 1;
}

// ███  $O(n)$  -- █████  $(n-1)/2$  ████
int ListDelete(SeqList *L, int i, int *e) {
    if (i < 1 || i > L->length) return 0;
```

```

    *e = L->elem[i - 1];
    for (int j = i; j < L->length; j++)
        L->elem[j - 1] = L->elem[j];
    L->length--;
    return 1;
}

// ██████████x████ 0(n) 0(1)
void DeleteAllX(SeqList *L, int x) {
    int k = 0;
    for (int i = 0; i < L->length; i++)
        if (L->elem[i] != x)
            L->elem[k++] = L->elem[i];
    L->length = k;
}

```

2.3 单链表

单链表用指针链接结点，顺序存取，插入/删除操作需先找前驱。

```

// ████████
typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;

// █████ 0(n) -- ████████
LinkList Create_Head(int a[], int n) {
    LinkList L = (LinkList)malloc(sizeof(LNode));
    L->next = NULL;
    for (int i = 0; i < n; i++) {
        LNode *s = (LNode*)malloc(sizeof(LNode));
        s->data = a[i];
        s->next = L->next;
        L->next = s;
    }
    return L;
}

// █████ 0(n) -- █████
LinkList Create_Tail(int a[], int n) {
    LinkList L = (LinkList)malloc(sizeof(LNode));
    L->next = NULL;
    LNode *r = L;
    for (int i = 0; i < n; i++) {
        LNode *s = (LNode*)malloc(sizeof(LNode));
        s->data = a[i];
        r->next = s;
        r = s;
    }
    r->next = NULL;
    return L;
}

// ████████ -- █████ 0(n)

```

```
void Reverse(LinkList L) {
    LNode *pre = NULL, *cur = L->next;
    while (cur) {
        LNode *next = cur->next;
        cur->next = pre;
        pre = cur;
        cur = next;
    }
    L->next = pre;
}
```

2.4 顺序表 vs 链表对比

对比维度	顺序表	链表
存取方式	随机存取 $O(1)$	顺序存取 $O(n)$
插入/删除	$O(n)$ (需移动)	$O(n)$ (找前驱) + $O(1)$
空间分配	预分配, 可能浪费	动态分配, 有指针开销
缓存友好	是 (连续存储)	否 (结点分散)

2.5 408易错点

- 插入/删除位置*i*是位序(1-based), 对应下标*i*-1----位序和下标混用是高频陷阱
- 链表操作中, 修改指针顺序很重要: 先连后断, 防止丢失后继
- 头结点不是首元结点, 头结点data域通常闲置

第3章 栈和队列

408真题年份：括号匹配、表达式求值

3.1 408真题引入

[2023年408真题] 用两个栈实现队列

经典大题的变形考查，均摊时间复杂度分析。

[2021年408真题] 括号匹配问题

栈的应用----验证括号是否正确匹配。

3.2 栈(LIFO)

```
// 栈的基本操作
#define MAXSIZE 100
typedef struct {
    int data[MAXSIZE];
    int top;           // 初始为-1
} SqStack;

void InitStack(SqStack *S) { S->top = -1; }
int StackEmpty(SqStack *S) { return S->top == -1; }
int Push(SqStack *S, int e) {
    if (S->top == MAXSIZE - 1) return 0;
    S->data[++(S->top)] = e;
    return 1;
}
int Pop(SqStack *S, int *e) {
    if (StackEmpty(S)) return 0;
    *e = S->data[(S->top)--];
    return 1;
}

// 括号匹配
int BracketMatch(char *str) {
    SqStack S; InitStack(&S);
    for (int i = 0; str[i]; i++) {
        if (str[i] == '(' || str[i] == '[' || str[i] == '{')
            Push(&S, str[i]);
        else if (str[i] == ')' || str[i] == ']' || str[i] == '}') {
            if (StackEmpty(&S)) return 0;
            char top; Pop(&S, &top);
            if ((str[i] == ')' && top != '(') ||
                (str[i] == ']' && top != '[') ||
                (str[i] == '}' && top != '{'))
                return 0;
        }
    }
    return StackEmpty(&S);
}
```

3.3 队列(FIFO)

```
// 队列(队列)
#define MAXSIZE 100
typedef struct {
    int data[MAXSIZE];
    int front;    // 队头
    int rear;     // 队尾(队列)
} SqQueue;

void InitQueue(SqQueue *Q) { Q->front = Q->rear = 0; }
int QueueEmpty(SqQueue *Q) { return Q->front == Q->rear; }
int QueueFull(SqQueue *Q) {
    return (Q->rear + 1) % MAXSIZE == Q->front;
}
int EnQueue(SqQueue *Q, int e) {
    if (QueueFull(Q)) return 0;
    Q->data[Q->rear] = e;
    Q->rear = (Q->rear + 1) % MAXSIZE;
    return 1;
}
int DeQueue(SqQueue *Q, int *e) {
    if (QueueEmpty(Q)) return 0;
    *e = Q->data[Q->front];
    Q->front = (Q->front + 1) % MAXSIZE;
    return 1;
}

// 队列
// 1. 队列(rear+1)%MAXSIZE == front
// 2. 队列 size == MAXSIZE
// 3. 队列 tag == front == rear && tag == 1
```

3.4 双栈模拟队列(408高频)

```
// 双栈 -- 408
typedef struct {
    SqStack s1; // 栈1
    SqStack s2; // 栈2
} MyQueue;

void EnQueue(MyQueue *Q, int e) {
    Push(&Q->s1, e);
}
int DeQueue(MyQueue *Q, int *e) {
    if (StackEmpty(&Q->s2)) {
        int t;
        while (!StackEmpty(&Q->s1)) {
            Pop(&Q->s1, &t);
            Push(&Q->s2, t);
        }
    }
    if (StackEmpty(&Q->s2)) return 0;
    Pop(&Q->s2, e);
    return 1;
}
```

```
}  
// 0(1) -- 2, 2
```

3.5 408易错点

- 循环队列 $\text{front} == \text{rear}$ 分不清是空还是满，需额外手段(牺牲单元/size/tag)
- 链队出队时若删除最后一个结点，须同时更新 rear 指向头结点
- 表达式求值中，操作符栈 + 操作数栈，优先级比较是核心

第4章 串

408真题年份：KMP算法大题必考

4.1 408真题引入

[2015年408真题] KMP算法求next数组

给定模式串，手工计算next数组并完成匹配过程。

[2019年408真题] 朴素匹配与KMP对比

分析两种模式匹配算法的时间复杂度差异。

4.2 串的存储结构

注意：408中串的数组下标从1开始(ch[0]闲置)，与普通C编程不同。

```
// 串的定义
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN]; // 1~MAXLEN
    int length;
} SString;
```

4.3 朴素模式匹配

```
// 朴素模式匹配 O(mn)
int Index_BF(SString S, SString T) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) { i++; j++; }
        else {
            i = i - j + 2; // i回退
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

4.4 KMP算法(408大题必考)

核心思想：利用已匹配信息，主串指针i不回溯，只移动j到next[j]。

```
// KMP -- next数组 O(m)
void get_next(SString T, int next[]) {
    int j = 1, k = 0;
    next[1] = 0;
    while (j < T.length) {
        if (k == 0 || T.ch[j] == T.ch[k]) {
            k++;
            next[j] = k;
        }
        j++;
    }
}
```

```

        j++; k++;
        next[j] = k;
    } else {
        k = next[k];
    }
}
}

// KMP O(n)
int Index_KMP(SString S, SString T, int next[]) {
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) { i++; j++; }
        else { j = next[j]; }
    }
    if (j > T.length) return i - T.length;
    return 0;
}

```

4.5 Next数组手工求法

手算三步法：

第1步：写出模式串，编号 $j=1,2,\dots,m$

第2步：对每个 j ，观察 $T[1..j-1]$ 的最长相等前后缀长度 k

第3步： $next[j] = k+1$ ($next[1]=0, next[2]=1$)

实例： $T = \text{"abaabc"}$

j	1	2	3	4	5	6
T	a	b	a	a	b	c
next[j]	0	1	1	2	2	3

4.6 Nextval数组(KMP优化)

```

// nextval O(n)
void get_nextval(SString T, int nextval[]) {
    int j = 1, k = 0;
    nextval[1] = 0;
    while (j < T.length) {
        if (k == 0 || T.ch[j] == T.ch[k]) {
            j++; k++;
            if (T.ch[j] != T.ch[k])
                nextval[j] = k;
            else
                nextval[j] = nextval[k];
        } else {
            k = nextval[k];
        }
    }
}

```

4.7 408易错点

- 408串下标从1开始，ch[0]闲置----和标准C的strlen不同
- next[1]恒为0(不是-1！ 注意与某些教材的区别)
- next[2]恒为1----无论前两个字符是什么
- KMP不一定比朴素快：字符集很小的二进制串场景可能KMP常数更大

第5章 树与二叉树

408真题年份：遍历、BST删除、AVL旋转

5.1 408真题引入

[2023年408真题] 二叉树的遍历与构造

给定遍历序列构造二叉树，结合BST性质求解。

[2022年408真题] AVL树的插入与旋转

手动模拟AVL树的插入过程和四种旋转操作。

[2021年408真题] BST删除操作

BST删除三种情况的完整手算推导。

5.2 二叉树定义与性质

二叉树每个结点最多有两棵子树，且子树有左右之分。

性质	公式	408考频
第 <i>i</i> 层最多结点数	$2^{(i-1)}$	高
深度 <i>k</i> 的最大结点数	$2^k - 1$	高
叶子与度为2结点	$n_0 = n_2 + 1$	极高
完全二叉树深度	$\text{floor}(\log_2 n) + 1$	高
完全二叉树编号	左 $2i$, 右 $2i+1$, 父 $\text{floor}(i/2)$	极高

5.3 二叉树的链式存储与遍历

```
// ■■■■■■■■
typedef char ElemType;
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;

// ■■■■■■■■ -> ■ -> ■
void PreOrder(BiTree T) {
    if (T == NULL) return;
    visit(T); PreOrder(T->lchild); PreOrder(T->rchild);
}

// ■■■■■■■■ -> ■ -> ■
void InOrder(BiTree T) {
    if (T == NULL) return;
```

```

        InOrder(T->lchild); visit(T); InOrder(T->rchild);
    }
    // 中序遍历二叉树
void PostOrder(BiTree T) {
    if (T == NULL) return;
    PostOrder(T->lchild); PostOrder(T->rchild); visit(T);
}

// 中序遍历二叉树(408)
void InOrder_NonRecursive(BiTree T) {
    BiTNode *stack[100]; int top = -1;
    BiTNode *p = T;
    while (p != NULL || top != -1) {
        if (p != NULL) {
            stack[++top] = p;
            p = p->lchild;
        } else {
            p = stack[top--];
            visit(p);
            p = p->rchild;
        }
    }
}

// 层次遍历二叉树
void LevelOrder(BiTree T) {
    if (T == NULL) return;
    BiTNode *queue[100];
    int front = 0, rear = 0;
    queue[rear++] = T;
    while (front != rear) {
        BiTNode *p = queue[front++];
        visit(p);
        if (p->lchild) queue[rear++] = p->lchild;
        if (p->rchild) queue[rear++] = p->rchild;
    }
}

```

5.4 二叉排序树 BST

BST性质：左子树所有结点值 < 根 < 右子树所有结点值。中序遍历BST得到递增有序序列。

```

// BST
typedef struct BSTNode {
    int key;
    struct BSTNode *lchild, *rchild;
} BSTNode, *BSTree;

// BST
BSTNode* BST_Search(BSTree T, int key) {
    while (T != NULL && key != T->key) {
        if (key < T->key) T = T->lchild;
        else T = T->rchild;
    }
    return T;
}

```



```

}

// BST■■■
bool BST_Insert(BSTree *T, int key) {
    if (*T == NULL) {
        *T = (BSTNode*)malloc(sizeof(BSTNode));
        (*T)->key = key;
        (*T)->lchild = (*T)->rchild = NULL;
        return true;
    }
    if (key == (*T)->key) return false;
    if (key < (*T)->key)
        return BST_Insert(&((*T)->lchild), key);
    else
        return BST_Insert(&((*T)->rchild), key);
}

// BST■■■ -- ■■■■■
bool BST_Delete(BSTree *T, int key) {
    if (*T == NULL) return false;
    if (key < (*T)->key) return BST_Delete(&(*T)->lchild, key);
    if (key > (*T)->key) return BST_Delete(&(*T)->rchild, key);
    BSTNode *del = *T;
    if ((*T)->lchild == NULL) { *T = (*T)->rchild; free(del); }
    else if ((*T)->rchild == NULL) { *T = (*T)->lchild; free(del); }
    else {
        BSTNode *succ = (*T)->rchild;
        BSTNode *succ_parent = *T;
        while (succ->lchild != NULL) {
            succ_parent = succ; succ = succ->lchild;
        }
        (*T)->key = succ->key;
        if (succ_parent == *T) succ_parent->rchild = succ->rchild;
        else succ_parent->lchild = succ->rchild;
        free(succ);
    }
    return true;
}

```

5.5 平衡二叉树 AVL

AVL树：任意结点左右子树高度差不超过1的BST。平衡因子 $BF = \text{左高} - \text{右高} \in \{-1, 0, 1\}$

```

// AVL■■■■■
typedef struct AVLNode {
    int key;
    int bf; // ■■■■■
    struct AVLNode *lchild, *rchild;
} AVLNode, *AVLTree;

// LL ■■■■
AVLNode* Rotate_LL(AVLNode *A) {
    AVLNode *B = A->lchild;
    A->lchild = B->rchild;
    B->rchild = A;
}

```

```

        return B;
    }
    // RR ■■■■
    AVLNode* Rotate_RR(AVLNode *A) {
        AVLNode *B = A->rchild;
        A->rchild = B->lchild;
        B->lchild = A;
        return B;
    }
    // LR ■■■■■■
    AVLNode* Rotate_LR(AVLNode *A) {
        A->lchild = Rotate_RR(A->lchild);
        return Rotate_LL(A);
    }
    // RL ■■■■■■
    AVLNode* Rotate_RL(AVLNode *A) {
        A->rchild = Rotate_LL(A->rchild);
        return Rotate_RR(A);
    }
}

```

5.6 旋转判定速查

插入场景	失衡BF	旋转
LL (左子树左子树)	+2	右单旋(LL)
RR (右子树右子树)	-2	左单旋(RR)
LR (左子树右子树)	+2	先左后右(LR)
RL (右子树左子树)	-2	先右后左(RL)

5.7 堆

```

// ■■■■■■
#define MAX_HEAP 100
typedef struct {
    int data[MAX_HEAP];
    int size;
} MaxHeap;

// ■■ -- ■■■■■■
void Heap_Up(MaxHeap *H, int k) {
    while (k > 1 && H->data[k] > H->data[k/2]) {
        int t = H->data[k]; H->data[k] = H->data[k/2]; H->data[k/2] = t;
        k = k / 2;
    }
}

// ■■ -- ■■■■■■■■
void Heap_Down(MaxHeap *H, int k) {
    while (2 * k <= H->size) {
        int j = 2 * k;
        if (j < H->size && H->data[j] < H->data[j+1]) j++;
        if (H->data[k] >= H->data[j]) break;
    }
}

```

```

        int t = H->data[k]; H->data[k] = H->data[j]; H->data[j] = t;
        k = j;
    }
}

// ■■■ O(n)

void BuildHeap(MaxHeap *H, int arr[], int n) {
    H->size = n;
    for (int i = 1; i <= n; i++) H->data[i] = arr[i-1];
    for (int i = n/2; i >= 1; i--) Heap_Down(H, i);
}

```

5.8 哈夫曼树与哈夫曼编码

```
// ■■■■■■(■■■■■)
typedef struct {
    int weight;
    int parent, lchild, rchild;
} HTNode;

// ■■■■■■
void CreateHuffmanTree(HTNode HT[], int w[], int n) {
    int m = 2 * n - 1;
    for (int i = 1; i <= m; i++) {
        HT[i].parent = HT[i].lchild = HT[i].rchild = 0;
        HT[i].weight = (i <= n) ? w[i-1] : 0;
    }
    for (int i = n + 1; i <= m; i++) {
        int s1 = 0, s2 = 0, min1 = INT_MAX, min2 = INT_MAX;
        for (int j = 1; j < i; j++) {
            if (HT[j].parent == 0) {
                if (HT[j].weight < min1) {
                    min2 = min1; s2 = s1;
                    min1 = HT[j].weight; s1 = j;
                } else if (HT[j].weight < min2) {
                    min2 = HT[j].weight; s2 = j;
                }
            }
        }
        HT[s1].parent = HT[s2].parent = i;
        HT[i].lchild = s1; HT[i].rchild = s2;
        HT[i].weight = HT[s1].weight + HT[s2].weight;
    }
}
```

5.9 408易错点

- $n_0 = n_2 + 1$ 对所有非空二叉树成立，不仅限于完全/满二叉树
- 度为2的树 != 二叉树(二叉树区分左右次序)
- 完全二叉树判定：某结点只有右孩子无左孩子 -> 必非完全二叉树
- 建堆是 $O(n)$ ，不是 $O(n \log n)$ ----408常设陷阱
- 哈夫曼树 $n_1=0$ ，总结点数= $2n-1$ (n 为叶子数)

- 先序+后序不能唯一确定二叉树，需中序

第6章 图

408真题年份：Dijkstra、拓扑排序、关键路径

6.1 408真题引入

[2023年408真题] Dijkstra求最短路径

给定图，手工模拟Dijkstra算法的完整执行过程。

[2022年408真题] 拓扑排序与关键路径

给定AOE网，计算 $ve/vl/e/l$ ，找出关键路径。

[2021年408真题] DFS/BFS遍历

非连通图的遍历，调用DFS的次数等于连通分量数。

6.2 图的存储结构

```
// ■■■■
#define MAXV 100
#define INF INT_MAX
typedef struct {
    int vexnum, arcnum;
    int edges[MAXV][MAXV];
} MGraph;

// ■■■
typedef struct ArcNode {
    int adjvex;
    struct ArcNode *next;
} ArcNode;
typedef struct VNode {
    char data;
    ArcNode *first;
} VNode;
typedef struct {
    VNode vertices[MAXV];
    int vexnum, arcnum;
} ALGraph;
```

6.3 DFS与BFS遍历

```
int visited[MAXV];

// DFS(■■■■■)
void DFS_MGraph(MGraph G, int v) {
    printf("%d ", v); visited[v] = 1;
    for (int j = 0; j < G.vexnum; j++)
        if (G.edges[v][j] != INF && G.edges[v][j] != 0 && !visited[j])
            DFS_MGraph(G, j);
}
```

```

}

// BFS(■■■■■)
void BFS_ALGraph(ALGraph G, int v) {
    int q[MAXV], front = 0, rear = 0;
    printf("%d ", v); visited[v] = 1; q[rear++] = v;
    while (front != rear) {
        int u = q[front++];
        ArcNode *p = G.vertices[u].first;
        while (p) {
            if (!visited[p->adjvex]) {
                printf("%d ", p->adjvex);
                visited[p->adjvex] = 1;
                q[rear++] = p->adjvex;
            }
            p = p->next;
        }
    }
}

// ■■■■■■
void DFSTraverse(MGraph G) {
    for (int i = 0; i < G.vexnum; i++) visited[i] = 0;
    for (int i = 0; i < G.vexnum; i++)
        if (!visited[i]) DFS_MGraph(G, i);
}

```

6.4 Dijkstra最短路径

```

// Dijkstra■■(■■■■■)
void Dijkstra(MGraph G, int v0) {
    int dist[MAXV], path[MAXV], S[MAXV] = {0};
    for (int i = 0; i < G.vexnum; i++) {
        dist[i] = G.edges[v0][i];
        path[i] = (G.edges[v0][i] < INF && i != v0) ? v0 : -1;
    }
    S[v0] = 1; dist[v0] = 0;
    for (int k = 1; k < G.vexnum; k++) {
        int minDist = INF, u = -1;
        for (int i = 0; i < G.vexnum; i++)
            if (!S[i] && dist[i] < minDist) { minDist = dist[i]; u = i; }
        if (u == -1) break;
        S[u] = 1;
        for (int w = 0; w < G.vexnum; w++)
            if (!S[w] && G.edges[u][w] != INF && G.edges[u][w] != 0
                && dist[u] + G.edges[u][w] < dist[w]) {
                dist[w] = dist[u] + G.edges[u][w];
                path[w] = u;
            }
    }
}

```

6.5 拓扑排序

```
// 拓扑排序( Kahn's Algorithm )
int TopologicalSort(ALGraph G, int topo[]) {
    int indegree[MAXV] = {0};
    int stack[MAXV], top = -1, count = 0;
    for (int i = 0; i < G.vexnum; i++) {
        ArcNode *p = G.vertices[i].first;
        while (p) { indegree[p->adjvex]++; p = p->next; }
    }
    for (int i = 0; i < G.vexnum; i++)
        if (indegree[i] == 0) stack[++top] = i;
    while (top != -1) {
        int u = stack[top--]; topo[count++] = u;
        ArcNode *p = G.vertices[u].first;
        while (p) {
            int w = p->adjvex;
            indegree[w]--;
            if (indegree[w] == 0) stack[++top] = w;
            p = p->next;
        }
    }
    if (count < G.vexnum) return 0;
    return 1;
}
```

6.6 关键路径(AOE网)

四个关键量：ve(最早发生时间), vl(最迟发生时间), e(活动最早开始), l(活动最迟开始)。关键活动：e(i)=l(i)。

ve从源点向前递推(取max)，vl从汇点向后递推(取min)。

注意：ve取max，vl取min----方向记反则全错！

6.7 最小生成树

算法	策略	时间复杂度	适合
Prim	选顶点（离集合最近）	$O(n^2)$	稠密图
Kruskal	选边（权值最小）	$O(e \log e)$	稀疏图

6.8 Floyd算法

```
// Floyd -- 求所有点对的最短路径
void Floyd(MGraph G, int dist[][MAXV], int path[][MAXV]) {
    for (int i = 0; i < G.vexnum; i++)
        for (int j = 0; j < G.vexnum; j++) {
            dist[i][j] = G.edges[i][j];
            path[i][j] = (G.edges[i][j] < INF) ? i : -1;
        }
    for (int k = 0; k < G.vexnum; k++)
        for (int i = 0; i < G.vexnum; i++)
            for (int j = 0; j < G.vexnum; j++)
```

```
        if (dist[i][k] != INF && dist[k][j] != INF
            && dist[i][k] + dist[k][j] < dist[i][j]) {
            dist[i][j] = dist[i][k] + dist[k][j];
            path[i][j] = path[k][j];
        }
    }
```

6.9 408易错点

- Dijkstra不允许负权边(已确定最短距离的顶点可能被负权边再次缩短)
- Floyd三重循环k必须在最外层----顺序错误导致结果错误
- ve取max, vl取min----方向记反则全错
- 调用DFS/BFS的次数=连通分量数
- BFS求最短路径仅当无权图(或所有边权相等)时才成立

第7章 查找

408真题年份：ASL计算、B树分裂、哈希表

7.1 408真题引入

[2023年408真题] B树的插入与分裂

5阶B树插入关键字序列，手动模拟分裂过程。

[2022年408真题] 哈希表ASL计算

线性探测法处理冲突，计算查找成功/失败的ASL。

[2020年408真题] 二分查找判定树

画出二分查找的判定树，计算ASL。

7.2 二分查找

```
// 二分查找
int BinarySearch(int arr[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] > key) hi = mid - 1;
        else lo = mid + 1;
    }
    return -1;
}
```

7.3 顺序查找(哨兵优化)

```
// 哨兵优化
int SeqSearch_Sentinel(int arr[], int n, int key) {
    arr[0] = key;
    int i;
    for (i = n; arr[i] != key; i--);
    return i;
}
```

7.4 B树(408大题重点)

m阶B树：根结点最少1个关键字，非根内部结点最少 $\lceil m/2 \rceil - 1$ 个，最多m-1个。所有叶结点在同一层。

```
// B树定义
#define M 5
#define MAX_KEY (M-1)
#define MIN_KEY ((M+1)/2 - 1)
```

```
typedef struct BTreeNode {
    int keys[M];
    struct BTreeNode *children[M+1];
    int keyNum;
    bool isLeaf;
} BTreeNode;

// B■■■■
BTreeNode* BTree_Search(BTreeNode *root, int key) {
    if (root == NULL) return NULL;
    int i = 1;
    while (i <= root->keyNum && key > root->keys[i]) i++;
    if (i <= root->keyNum && root->keys[i] == key) return root;
    if (root->isLeaf) return NULL;
    return BTree_Search(root->children[i-1], key);
}
```

7.5 哈希表

常用哈希函数：除留余数法 $H(key) = key \% p$ (p 取不大于表长的最大素数)。

```
// ██████████  
#define TABLE_SIZE 13  
  
typedef struct {  
    int key;  
    bool occupied;  
    bool deleted;  
} HashTable;  
  
HashTable ht[TABLE_SIZE];  
  
int HashInsert_Linear(int key) {  
    int hash = key % TABLE_SIZE;  
    for (int d = 0; d < TABLE_SIZE; d++) {  
        int addr = (hash + d) % TABLE_SIZE;  
        if (!ht[addr].occupied || ht[addr].deleted) {  
            ht[addr].key = key;  
            ht[addr].occupied = true;  
            ht[addr].deleted = false;  
            return 1;  
        }  
    }  
    return 0;  
}  
  
// ████████████████████  
bool HashDelete_Linear(int key) {  
    int addr = HashSearch_Linear(key);  
    if (addr == -1) return false;  
    ht[addr].deleted = true;  
    return true;  
}
```

7.6 ASL计算示例

装填因子 $\alpha = n/m$ 。ASL成功 = (各元素探测次数和)/ n 。

例：表长 $m=13$ ， $H(key)=key\%13$ ，线性探测。插入序列：19,14,23,01,68,20,84,27,55,11,10,79。ASL成功 $=30/12=2.5$ 。

7.7 408易错点

- 二分查找必须是有序顺序表(链表不支持随机存取)
- 哈希表开放定址法删除必须打标记，不能直接置空
- B树根结点没有最少关键字限制(可以只有1个)
- B+树内部结点不存数据，查找必须到叶结点
- 装填因子增大 $\rightarrow$ 冲突增多 $\rightarrow$ ASL增大
- 分块查找块内只能顺序查找，索引表可以二分查找
- B树删除借兄弟时，被借关键字通过父结点中转，不是直接移过去

第8章 排序

408真题年份：快排、堆排、归并、基数

8.1 408真题引入

[2023年408真题] 快速排序划分过程

给定序列，手写Partition过程，分析快排递归深度。

[2022年408真题] 堆排序建堆与调整

给定无序序列，建大根堆并模拟堆排序全过程。

[2021年408真题] 归并排序合并过程

两个有序序列合并，分析归并排序空间复杂度。

8.2 插入排序

```
// ■■■■■■  $O(n^2)$ 
void InsertSort(int A[], int n) {
    int i, j, temp;
    for (i = 1; i < n; i++) {
        if (A[i] < A[i-1]) {
            temp = A[i];
            for (j = i-1; j >= 0 && A[j] > temp; j--)
                A[j+1] = A[j];
            A[j+1] = temp;
        }
    }
}

// ■■■■ -- ■■■■  $O(n^{1.3})$ 
void ShellSort(int A[], int n) {
    int gap, i, j, temp;
    for (gap = n/2; gap > 0; gap /= 2) {
        for (i = gap; i < n; i++) {
            if (A[i] < A[i-gap]) {
                temp = A[i];
                for (j = i-gap; j >= 0 && A[j] > temp; j -= gap)
                    A[j+gap] = A[j];
                A[j+gap] = temp;
            }
        }
    }
}
```

8.3 交换排序

```
// ■■■■ -- ■■  $O(n) \sim O(n^2)$ 
void BubbleSort(int A[], int n) {
    int i, j, temp; bool flag;
```

```

    for (i = 0; i < n-1; i++) {
        flag = false;
        for (j = n-1; j > i; j--) {
            if (A[j-1] > A[j]) {
                temp = A[j]; A[j] = A[j-1]; A[j-1] = temp;
                flag = true;
            }
        }
        if (!flag) break;
    }
}
// ■■■■ -- ■■■ O(n log n)
int Partition(int A[], int low, int high) {
    int pivot = A[low];
    while (low < high) {
        while (low < high && A[high] >= pivot) high--;
        A[low] = A[high];
        while (low < high && A[low] <= pivot) low++;
        A[high] = A[low];
    }
    A[low] = pivot;
    return low;
}
void QuickSort(int A[], int low, int high) {
    if (low < high) {
        int pivotPos = Partition(A, low, high);
        QuickSort(A, low, pivotPos - 1);
        QuickSort(A, pivotPos + 1, high);
    }
}

```

8.4 选择排序

```

// ■■■■■■ -- ■■■ O(n^2)
void SelectSort(int A[], int n) {
    int i, j, minIdx, temp;
    for (i = 0; i < n-1; i++) {
        minIdx = i;
        for (j = i+1; j < n; j++)
            if (A[j] < A[minIdx]) minIdx = j;
        if (minIdx != i) {
            temp = A[i]; A[i] = A[minIdx]; A[minIdx] = temp;
        }
    }
}
// ■■■ -- ■■■ O(n log n)
void HeadAdjust(int A[], int k, int n) {
    int rootVal = A[k];
    for (int i = 2*k+1; i < n; i = 2*i+1) {
        if (i+1 < n && A[i] < A[i+1]) i++;
        if (rootVal >= A[i]) break;
        A[k] = A[i]; k = i;
    }
    A[k] = rootVal;
}

```

```

void BuildMaxHeap(int A[], int n) {
    for (int i = n/2-1; i >= 0; i--) HeadAdjust(A, i, n);
}
void HeapSort(int A[], int n) {
    BuildMaxHeap(A, n);
    for (int i = n-1; i > 0; i--) {
        int temp = A[0]; A[0] = A[i]; A[i] = temp;
        HeadAdjust(A, 0, i);
    }
}

```

8.5 归并排序

```

// ■■■■■■ -- ■■ O(n log n)
int *B = NULL;
void Merge(int A[], int low, int mid, int high) {
    for (int k = low; k <= high; k++) B[k] = A[k];
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        if (B[i] <= B[j]) A[k++] = B[i++];
        else A[k++] = B[j++];
    }
    while (i <= mid) A[k++] = B[i++];
    while (j <= high) A[k++] = B[j++];
}
void MergeSort(int A[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        MergeSort(A, low, mid);
        MergeSort(A, mid + 1, high);
        Merge(A, low, mid, high);
    }
}

```

8.6 八大排序汇总对比

算法	平均	最坏	空间	稳定
直接插入	$O(n^2)$	$O(n^2)$	$O(1)$	是
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(1)$	否
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	是
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	否
简单选择	$O(n^2)$	$O(n^2)$	$O(1)$	否
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	否
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

8.7 408易错点

- 快排空间复杂度是 $O(\log n)$ (递归栈), 不是 $O(1)$ ----408高频陷阱
- 建堆复杂度是 $O(n)$, 不是 $O(n \log n)$
- 堆排序不稳定(建堆和调整时跳跃式交换)
- 希尔排序不稳定(分组跳跃式移动)
- 归并排序需要 $O(n)$ 辅助空间, 是 $O(n \log n)$ 中唯一非原地算法
- 折半插入只减少比较次数, 移动次数仍为 $O(n^2)$

附录A 数据结构大题模板

408手写算法必背模板，建议考前熟练默写。

模板1：链表原地逆置

408出现概率最高的代码题之一

```
void Reverse(LinkList L) {
    LNode *pre = NULL, *cur = L->next, *next;
    while (cur != NULL) {
        next = cur->next; // ■■■■
        cur->next = pre;  // ■■■■
        pre = cur;
        cur = next;
    }
    L->next = pre;
}
// O(n), O(1)
```

模板2：删除链表中所有值为x的结点

带头结点的删除操作

```
void DeleteX(LinkList L, int x) {
    LNode *p = L, *q;
    while (p->next != NULL) {
        if (p->next->data == x) {
            q = p->next; p->next = q->next; free(q);
        } else { p = p->next; }
    }
}
// O(n), O(1)
```

模板3：两个有序链表归并

尾插法实现递增归并

```
LinkList Merge(LinkList A, LinkList B) {
    LinkList C = malloc(sizeof(LNode)); C->next = NULL;
    LNode *pa = A->next, *pb = B->next, *pc = C;
    while (pa && pb) {
        if (pa->data <= pb->data) {
            pc->next = pa; pa = pa->next;
        } else { pc->next = pb; pb = pb->next; }
        pc = pc->next;
    }
    pc->next = pa ? pa : pb;
    return C;
}
```



```
// O(m+n), O(1)
```

模板4：二叉树递归遍历

先序/中序/后序三件套

```
// ■■
void PreOrder(BiTree T) {
    if (T) { visit(T); PreOrder(T->lchild); PreOrder(T->rchild); }
}
// ■■
void InOrder(BiTree T) {
    if (T) { InOrder(T->lchild); visit(T); InOrder(T->rchild); }
}
// ■■
void PostOrder(BiTree T) {
    if (T) { PostOrder(T->lchild); PostOrder(T->rchild); visit(T); }
}
// O(n), O(h)
```

模板5：二叉树非递归中序

408大题极高频

```
void InOrder2(BiTree T) {
    SqStack S; InitStack(&S);
    BiTree p = T;
    while (p || !StackEmpty(&S)) {
        if (p) { Push(&S, p); p = p->lchild; }
        else { Pop(&S, &p); visit(p); p = p->rchild; }
    }
}
// O(n), O(h)
```

模板6：层序遍历

队列实现

```
void LevelOrder(BiTree T) {
    SqQueue Q; InitQueue(&Q);
    EnQueue(&Q, T);
    while (!QueueEmpty(&Q)) {
        BiTree p; DeQueue(&Q, &p);
        visit(p);
        if (p->lchild) EnQueue(&Q, p->lchild);
        if (p->rchild) EnQueue(&Q, p->rchild);
    }
}
// O(n), O(w)
```

模板7：顺序表删除所有x(双指针)

O(n)时间, O(1)空间经典技巧

```
int DeleteX(SeqList *L, int x) {
    int k = 0;
    for (int i = 0; i < L->length; i++)
        if (L->elem[i] != x)
            L->elem[k++] = L->elem[i];
    L->length = k;
    return 1;
}
// O(n), O(1)
```

模板8：二分查找

防溢出写法

```
int BinarySearch(int a[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
// O(log n)
```

模板9：KMP的next数组手算

三步法

```
// ■■■■
// 1. next[1]=0, next[2]=1
// 2. ■j>=3■■■P[1..j-1]■■■■■■■■■■+1
// ■■■■P="abaabc"
// j=1:0 j=2:1 j=3:1 j=4:2 j=5:2 j=6:3
```

408大题应试策略

策略	说明
先写注释思路	先写 // 思路：双指针... 再写代码，思路对拿过程分
注意边界条件	链表判NULL、循环队列判满判空、数组下标不越界
复杂度必标	每个函数末尾写 // O(n), O(1)

策略	说明
先保正确再优化	$O(n^2)$ 暴力的过程分 > $O(n)$ 优化但写错的0分
时间分配	代码题15分钟：5分钟思路+8分钟写代码+2分钟检查

附录B 数据结构复杂度速查表

各结构/算法时空复杂度总表，408高频陷阱汇总。

B.1 线性结构

操作	顺序表	单链表	循环队列	栈
按位查找	$O(1)$	$O(n)$	-	-
按值查找	$O(n)$	$O(n)$	-	-
插入	$O(n)$	$O(n)$	$O(1)$	$O(1)$
删除	$O(n)$	$O(n)$	$O(1)$	$O(1)$

B.2 树与二叉树

操作	二叉树	BST	AVL	堆
查找	$O(n)$	$O(h)$	$O(\log n)$	$O(n)$
插入	-	$O(h)$	$O(\log n)$	$O(\log n)$
删除	-	$O(h)$	$O(\log n)$	$O(\log n)$
建树/堆	-	$O(n \log n)$	$O(n \log n)$	$O(n)$

B.3 图

算法	邻接矩阵	邻接表
DFS	$O(n^2)$	$O(n+e)$
BFS	$O(n^2)$	$O(n+e)$
Prim	$O(n^2)$	$O(e \log n)$
Dijkstra	$O(n^2)$	$O(e \log n)$
Floyd	$O(n^3)$	-
拓扑排序	$O(n^2)$	$O(n+e)$

B.4 查找算法

算法	平均时间	最坏时间	空间	适用条件
顺序查找	$O(n)$	$O(n)$	$O(1)$	无序/链表均可

算法	平均时间	最坏时间	空间	适用条件
二分查找	$O(\log n)$	$O(\log n)$	$O(1)$	有序顺序表
分块查找	$O(\sqrt{n})$	$O(n)$	$O(1)$	分块有序
BST查找	$O(\log n)$	$O(n)$	$O(1)$	-
AVL查找	$O(\log n)$	$O(\log n)$	$O(1)$	-
B树查找	$O(\log n)$	$O(\log n)$	$O(1)$	外存数据
哈希表	$O(1)$	$O(n)$	$O(n)$	-

B.5 排序算法

算法	最好	平均	最坏	空间	稳定
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定

B.6 408高频陷阱

陷阱	正解
快排空间复杂度	$O(\log n)$ (递归栈), 不是 $O(1)$
堆排序稳定性	不稳定——建堆过程破坏相对顺序
建堆复杂度	$O(n)$, 不是 $O(n \log n)$
希尔排序稳定性	不稳定——跳跃式交换
归并排序空间	$O(n)$, 需要辅助数组
BST最坏情况	退化成链表→ $O(n)$, 不是 $O(\log n)$

-- 全文完 --

祝大家408数据结构取得好成绩！